

optimization.py

```
1  # Import packages
2  import os
3  import numpy as np
4  from pyslsqp import optimize as pyslsqp_optimize
5
6  # Import Functions
7  from MyAPDLCall import RunAPDL
8  from opt_logger import OptimizationLogger
9  from Post_Process import PostProcessor
10 from aggregate import ConstraintAggregate
11 from acs import ACSclass
12
13 # Optimization Function
14 def run_optimization(mapdl, opti_settings, var, Misc, Solver_Settings):
15
16     # Set active variables
17     active = [(name, data["bounds"]) for name, data in var.items() if data.get("active", True)]
18     names = [name for name, _ in active]
19     Misc["active_vars"] = names
20
21     # Print active variables
22     print(f"{len(names)} Design Variables included: " +
23           ", ".join(f"x{i}={n}" for i, n in enumerate(names)), flush=True)
24
25     # Set initial guess, bounds and ranges to np array
26     x0 = np.array([var[name]["value"] for name, _ in active], dtype=float)
27     bounds = [bnds for _, bnds in active]
28     xl, xu = np.array(bounds).T
29
30     # Step Options for each variable (uniform step size for all, except rad)
31     fd_step = []
32
33     for name in names:
34         xval = var[name]["value"]
35         step = 1E-4 * xval
36         fd_step.append(step)
37
38     acs = ACSclass(
39         alpha=Solver_Settings.get("acs_alpha", 1.0),
40         c0 = 1.0,
41         enabled = Solver_Settings.get("use_acs", True)
42     )
43
44     # Logger Options
45     logger = OptimizationLogger(
46         x0=x0,
47         bounds=bounds,
48         method="PySLSQP",
49         var_names=names,
50         options={
51             "acc": Solver_Settings["acc"],
52             "finite_diff_abs_step": fd_step, # fd_step
53             "maxiter": Solver_Settings["maxiter"],
54             "Aggregate": Solver_Settings["Aggregate"],
55             "p_value": Solver_Settings["p_value"],
56             "rho_value": Solver_Settings["rho_value"],
57             "relaxation": Solver_Settings["relaxation"],
58         },
59         save_folder=Misc["save_folder"],
60     )
61
62     # Internal cache so RunAPDL is only executed once per unique x
```

optimization.py

```
63     cache = {"x": None, "f": None, "c": None, "util": None, "v_agg": None, "g_max": None}
64
65     # Helper
66     def arr(v):
67         return np.asarray(v, dtype=float).ravel()
68
69     # Model evaluation
70     def evaluate_model(x):
71         # Read variables and check
72         x = arr(x)
73         if cache["x"] is not None and np.array_equal(x, cache["x"]):
74             return cache["f"], cache["c"]
75
76         # Convert variables to dict again
77         var_dict = dict(zip(names, x))
78
79         # Run the Model
80         f, segment_masses = RunAPDL(mapdl, x, Misc, opti_settings)
81
82         # Initiate Post Processing
83         pp = PostProcessor(var_dict, Misc, opti_settings)
84
85         # Call Utilization Ratios
86         ##### FIX DOUBLE utils CALL !
87         utils = [
88             pp.Util_LB(),
89             pp.Util_NF(),
90             pp.Util_S(),
91             pp.Util_T(),
92             #pp.Util_BNS(),
93             pp.Util_BR(),
94             pp.Util_IN(),
95         ]
96         col_brace = [(1-arr(c), 1-arr(b)) for c,b in utils]
97
98         # Set up the constraints
99         c_util = np.concatenate([
100             *[v for pair in col_brace for v in pair],
101             1- arr(pp.Util_BS()[0]), # stress constraint here
102             1- arr(pp.Util_BS()[1])) # deflection constraint here
103
104         # Handle aggregation of constraints
105         agg = ConstraintAggregate(
106             method = Solver_Settings["Aggregate"],
107             p_value = Solver_Settings["p_value"],
108             rho_value = Solver_Settings["rho_value"],
109             relaxation = Solver_Settings["relaxation"],
110         )
111
112         c_util_agg, v_agg, g_max = agg.agg_output(c_util, c_scale=acs.c)
113
114         # Segment Masses Constraint
115         mass_limit = float(opti_settings["segment_mass_limit"])
116         mass_constraint = mass_limit - arr(segment_masses)
117
118         # Collect them together
119         c = np.concatenate([
120             np.atleast_1d(c_util_agg),
121             *map(arr, pp.Class_2()),
122             arr(pp.Eigenvalue_1()),
123             mass_constraint
124         ])
```

```

125
126     print(f"Constraint vector length: {len(c)}")
127
128     logger.log_evaluation(x, f, segment_masses, v_agg=v_agg, g_max=g_max, c_value=acs.c)
129
130     util_report = {
131         "Util_LB": pp.Util_LB(),
132         "Util_NF": pp.Util_NF(),
133         "Util_S": pp.Util_S(),
134         "Util_T": pp.Util_T(),
135         #"Util_BNS": pp.Util_BNS(),
136         "Util_BR": pp.Util_BR(),
137         "Util_IN": pp.Util_IN(),
138         "Util_BS_sig": pp.Util_BS()[0], # stress
139         "Util_BS_defl": pp.Util_BS()[1], # deflection
140     }
141
142     # Update design variables and constraints
143     cache.update(x=x.copy(), f=float(f), c=c, util = util_report, v_agg = v_agg, g_max=g_max)
144
145     return f, c
146
147 # Objective function call that only returns mass
148 def objective(x):
149     """Objective function for PySLSQP."""
150     f_val, _ = evaluate_model(x)
151     return f_val
152
153 # Constrain function call that only returns constraints
154 def constraints(x):
155     """Constraint vector for PySLSQP."""
156     _, c_val = evaluate_model(x)
157     return c_val
158
159 def iteration_callback(x):
160     logger.log_iteration(x)
161
162     if cache["v_agg"] is not None and cache["g_max"] is not None:
163         acs.update(cache["v_agg"], cache["g_max"])
164
165     util = cache.get("util", None)
166     if util is not None:
167         logger.log_utilization(util)
168
169 # Create Folder
170 os.makedirs(Misc["save_folder"], exist_ok=True)
171 save_filename = os.path.join(Misc["save_folder"], "pyslsqp_history.hdf5") # Save File
172 summary_filename = os.path.join(Misc["save_folder"], "slsqp_summary.out") # Summary File
173
174 # Run PySLSQP
175 result = pyslsqp_optimize(
176     x0=x0,
177     obj=objective,
178     con=constraints,
179     meq=0, # all constraints are inequalities
180     xl=xl,
181     xu=xu,
182     callback=iteration_callback,
183     finite_diff_rel_step=1E-3,
184     maxiter=Solver_Settings["maxiter"],
185     acc=Solver_Settings["acc"], # Objective Function Tolerance
186     iprint=2, # print iteration info

```

optimization.py

```
187         #obj_scaler=0.8,                # Scale objective function for stability
188         save_itr="all",                  # save major iterations
189         save_vars=[
190             "majiter",                    # major iteration
191             "x",                          # design variables
192             "objective",                  # objective function
193             "constraints",                # constraint vector
194             "optimality",                 # optimality
195             "feasibility",                # feasibility
196             "step",                       # step
197             "gradient",                   # gradient
198             "jacobian",                   # jacobian
199         ],
200         save_filename=save_filename,
201         summary_filename=summary_filename,
202         warm_start=False, # For restarting optimization from prior optimization runs
203         hot_start=False, # For restarting optimization from prior optimization runs without re-evaluating the
al point
204         load_filename=save_filename, # Filename for restart file
205         visualize=True,
206         visualize_vars=['objective', 'optimality', 'feasibility', 'x[0]'], # Extra: 'gradient[0]', 'constraint
, 'multipliers[0]', 'jacobian[0,0]'
207     )
208     # return the result, the text path, and the csv path
209     return result, logger.txt_path, logger.csv_path
```